Lab 4: VibeCoding — Coin Collector Game

Course: Software Engineering (SE) — Semester 5
Assignment: Lab 4 (VibeCoding)
Student Name: Shashank K
SRN: PES1UG24CS434
Section: Section H
Assigned Repository: 06_coin_collector (SETAPESU26)
Tool Used: Gemini / Claude (AI Pair Programming Assistant)

---

1. Executive Summary & Objective

The objective of Lab 4 (VibeCoding) is to utilize LLM-assisted pair programming and prompt engineering to diagnose broken game code, implement necessary bug fixes, and construct three key gameplay features within 3 to 4 prompt iterations.

The target project is a top-down **Coin Collector** game built using **Pygame**. The initial build suffered from a severe coin-collection bug where standing on or passing through a coin awarded points continuously on every frame without removing the coin.

Through systematic prompt engineering, four deliverables were achieved: 1. **Task 1:** Fixed the coin collection logic so coins score exactly once and are removed from play. 2. **Task 2:** Introduced three distinct coin types (Bronze, Silver, Gold) with varied values, colors, and radii. 3. **Task 3:** Implemented dynamic moving obstacles with a 3-lives system and a 1.5-second damage cooldown to prevent continuous damage. 4. **Task 4:** Added a 30-second round countdown timer, HUD display, Game Over / Round Over state, and an instant round restart mechanism (`[R]`).

---

2. Deliverables Checklist

- [x] **10-Second Before Video**: Demonstrating the infinite coin scoring bug (`Lab-4/videos/before_recording.webm`).
- [x] **10-Second After Video**: Demonstrating all bug fixes and features working in live gameplay (`Lab-4/videos/after_recording.webm`).
- [x] **Updated Codebase**: Complete Pygame code placed in `Lab-4/code/coin-collector/`.
- [x] **Vibe Coding Interaction History**: Documented below with exact prompts, diagnosis, code solutions, and test verification.

---

3. Detailed Prompt Iterations & Interaction History

Iteration 1: Repository Exploration, Bug Diagnosis & Baseline Video

User Prompt:

> *"Here is my assigned repository: `https://github.com/SETAPESU26/06_coin_collector`. Analyze the codebase, identify why coin collection is broken, and instruct me on how to run and record a 10-second 'before' video showing the bug before making any modifications."*

AI Assistant Response & Technical Diagnosis:

- **Repository Architecture:**
- `main.py`: Main game loop (event polling, engine update, renderer draw).
- `game/game_engine.py`: Owns player, coins, and updates score via `check_collection`.
- `game/coin.py`: Simple circle entity with bounding box `get_rect()`.
- `game/collection.py`: Hit-test collision checker between player and coin rects.
- `game/renderer.py`: Basic Pygame drawing routines.
- **Root Cause Analysis (Bug):** In `game/game_engine.py`: `python def update(self): collected = check_collection(self.player, self.coins) for coin in collected: self.score += coin.value` `check_collection()` detects intersection between the player's bounding box and each coin. However, `collected` coins were **never removed** from `self.coins`. At 60 FPS, passing through a single coin awarded 60 points per second indefinitely.
- **Action Taken:** Installed `pygame==2.6.1` into the Python 3.12 environment. Paused execution so the student could run `python main.py` and capture the **10-second 'before' video** showing the rapidly inflating score on a single coin.

Iteration 2: Bug Fix (Task 1 — Single-Touch Coin Collection)

User Prompt:

> *"Fix the coin collection bug so each coin awards points exactly once, disappears upon contact, and new coins respawn dynamically so the play area remains populated."*

AI Assistant Solution:

Refactored `game_engine.py`: 1. When `check_collection` returns intersecting coins, iterate and remove each coin from `self.coins`. 2. Add dynamic replenishment to keep the coin count at `NUM_COINS`:

```
# In game/game_engine.py
collected = check_collection(self.player, self.coins)
for coin in collected:
    self.score += coin.value
    self.coins.remove(coin)

while len(self.coins) < NUM_COINS:
    self.coins.append(self._random_coin())
```

Outcome: Coins are collected cleanly on contact, award points once, and replenish seamlessly.

Iteration 3: Entity Variety & Obstacle System (Task 2 & Task 3)

User Prompt:

> *"1. Add three distinct coin types: Bronze (1 point), Silver (3 points), and Gold (5 points) with clearly distinguishable colors and sizes.*
> *2. Add dynamic obstacles to the play area that damage the player. Implement a 3-lives system and an invulnerability period with visual feedback so one touch does not instantly drain all lives."*

AI Assistant Solution:

1. **Multiple Coin Types (`game/coin.py`):** Introduced a coin configuration dictionary and factory method:

2. **Bronze**: Value = 1 pt, Color = `(205, 127, 50)` (Bronze), Radius = 10 px.
3. **Silver**: Value = 3 pts, Color = `(192, 192, 192)` (Silver), Radius = 13 px.
4. **Gold**: Value = 5 pts, Color = `(255, 215, 0)` (Gold), Radius = 16 px.
5. Randomized weighted spawning: 50% Bronze, 35% Silver, 15% Gold.
6. **Obstacle Class (`game/obstacle.py`):** Created an autonomous obstacle that patrols the game window:
7. Velocity vectors (`vx`, `vy`) bouncing off boundaries.
8. Distinct hazard styling with danger crosses and red highlights.
9. Spawned outside the player's initial position (`safe_zone`).
10. **Lives & Invulnerability Handling (`game/game_engine.py` & `game/renderer.py`):**
11. Player initialized with 3 Lives (`self.lives = 3`).
12. On collision with an obstacle: `python if self.invulnerability_timer <= 0 and player_rect.colliderect(obstacle.get_rect()): self.lives -= 1 self.invulnerability_timer = 1.5 # 1.5-second cooldown if self.lives <= 0: self.lives = 0 self.game_over = True self.game_over_reason = "Out of Lives!"`
13. During the 1.5-second cooldown, the player sprite flickers rapidly to signal invulnerability and prevent unfair instant deaths.

Iteration 4: Timed Round & Game Over / Restart Flow (Task 4)

User Prompt:

> *"Implement a 30-second round countdown timer. Display the remaining time on the screen alongside score and lives. When time runs out or lives reach zero, stop the round, display the final score in a modal card, and let the player press [R] to restart a fresh round."*

AI Assistant Solution:

1. **High-Precision Delta Time (`main.py`):** Updated game loop to measure real elapsed seconds using `clock.tick(60) / 1000.0` and passed `dt` to `engine.update(dt)`.
2. **HUD Dashboard (`game/renderer.py`):**
3. Top translucent panel showing **Score** (left), **Timer** (center, turns red under 5 seconds), and **Lives** (right, heart icons).
4. **Round Conclusion & Game Over Modal:**
5. Triggers when `time_remaining <= 0` ("Time's Up!") or `lives <= 0` ("Out of Lives!").
6. Center modal displays final score, result header, and prompts:
    - `[R]`: Resets score, lives, timer (30.0s), and entity positions.
    - `[ESC]`: Quits the game.

---

4. Verification & Testing

An automated headless validation suite was executed using Pygame's dummy video driver:

```
pygame 2.6.1 (SDL 2.28.4, Python 3.12.3)
Initial score: 0
Initial lives: 3
Initial coins: 8
Initial obstacles: 3
```

```
Score after collecting 1 coin: 3 (Expected > 0)
Coin count maintained (respawn): 8
Score after 30 frames sitting still: 3 (Score before was: 3)
Lives after obstacle hit: 2 (Expected 2)
Invulnerability active: True
Lives after 30 frames overlapping obstacle: 2 (Expected still 2)
Game Over triggered on timeout: True Time's Up!
Reset: lives= 3 score= 0 time= 30.0 game_over= False
ALL VERIFICATIONS PASSED!
```

Additionally, manual gameplay verification was completed, and the **10-second 'after' video** was recorded showcasing: 1. One-touch coin collection across Bronze, Silver, and Gold coins. 2. Obstacle collision with life deduction and visual invulnerability flicker. 3. 30-second countdown timer counting down in real-time. 4. Game Over screen with score display and successful instant restart with `[R]`.

---

5. File Structure of Deliverables

```
mentorAlumniPlatform_PES1UG24CS434-LAB3-ShashankK/
└── Lab-4/
    ├── README.md                                  # Lab 4 overview and instructions
    ├── videos/
    │   ├── before_recording.webm                  # 10s gameplay before changes (showing bug)
    │   └── after_recording.webm                   # 10s gameplay after changes (all features working)
    ├── chat_history/
    │   ├── Lab_4_VibeCoding_Session_Report.md     # Full markdown vibe-coding prompt log
    │   └── Lab_4_VibeCoding_Session_Report.pdf    # Exported PDF report
    └── code/
        └── coin-collector/
            ├── main.py                            # Game loop & delta-time clock
            ├── requirements.txt                   # pygame>=2.5.0
            └── game/
                ├── __init__.py
                ├── coin.py                        # Bronze, Silver, Gold coin types
                ├── collection.py                  # Collision bookkeeping
                ├── game_engine.py                 # Core game state, timer, lives, reset
                ├── obstacle.py                    # Moving hazard obstacles
                ├── player.py                      # Player movement & bounds
                └── renderer.py                    # Graphics, HUD, Game Over modal
```